**1.** (Dominating Set in a Tree)

**算法设计** (只计算最小值):

```
function solve(u, T) :
    (c, d_r, d_o) ← (0, 0, −1)
    for v ∈ son_T(u) do
        (c', d'_r, d'_o) ← solve(v, T)
        (c, d_r, d_o) ← (c + c', max{d_r, d'_r + 1}, max{d_o, d'_o − 1})
    if d_o ≥ d_r then      //alldominated
        return (c, −1, d_o)
    elif d_r = k then      //choose u into S
        return (c + 1, −1, k)
    else     //waitingdominatoroutofsubtree
        return (c, d_r, d_o)
```

取定任意根 $r \in V$, 调用 $(c, d_r, d_o) \leftarrow \mathrm{solve}(u, T)$, 则 $c + [d_r \neq -1]$ 就是所求最小值.

**正确性分析**: 算法遵循如下贪心思路: 按子树递归地贪心, <u>只要子树内未被支配的结点有可能被子树外结点支配, 就不选取子树根作为支配点</u> $(*)$. 为此, 算法维护了:

- $c$, 表示完成子树内目前贪心支配策略所需的支配点数量;
- $d_r$, 表示子树内最深未支配点的深度;
- $d_o$, 表示子树内支配点能够跨越子树根向外支配的最远距离.

并讨论:

- 若 $d_o \geq d_r$ 时, 可以跨越根节点完成对子树内所有点的支配, 则 $d_r = -1$ (或 $-\infty$), $d_o$ 正常上传;
- 否则若 $d_r = k$, 必须选取子树根作为支配点, 不然 $d_r$ 对应的结点无法被支配. $c$ 增加, $d_r = -1$, $d_o = k$ (从子树根出发);
- 否则, 相安无事, 等待来自子树外的支配.

只需要证明 $(*)$ 断言的正确性. 不妨以 $r$ 为根, 对任意取到最优的 $S$, 将 $S$ 中的结点按深度降序排列为 $\{s_1, \cdots, s_k\}$. 依次枚举 $s_i$, 不断尝试令 $s_i \leftarrow \mathrm{father}(s_i)$ 并保持 $S$ 仍然是 $k$-支配集. 由深度关系可以看出, 完成变化的 $S$ 在删除重复元素后满足 $(*)$ 性质. 因此 $(*)$ 性质下总存在最优解.

**复杂度分析**: 明显 $\mathcal{O}(n)$.

**2.** (Minimum Spanning $k$-Forest)

(1) 对任意 $k$-forest, 令 $C = \{(u, v) : \forall S \in \Pi,\ u \notin \Pi \vee v \notin \Pi\} \cap E$, 由于 $j > k$, 必然 $|C| \neq \varnothing$. 对任意 minimum $k$-forest $F$, 如果边权最小 $(a, b) \in C \setminus F$, 则讨论:

- 若 $a, b$ 在 $F$ 中连通, 则任取 $(u, v) \in C \cap F$ 且 $u, v, a, b$ 在 $F$ 中两两连通,
- 否则任取 $(u, v) \in C \cap F$;

显然 $(u, v)$ 总是存在, $w(u, v) > w(a, b)$, 且在 $F$ 中总能用 $(a, b)$ 替换 $(u, v)$ 得到 $F'$, 使得 $F'$ 仍然是 $k$-forest. 这与 $F$ 的最小性矛盾.

(2) **算法设计**: 将 $E$ 按照边权升序排列, 依次枚举 $(u, v) \in E$, 只要 $(u, v)$ 加入时不构成环, 则将 $(u, v)$ 加入, 直到已然构成 $k$-forest.

**正确性分析**: 令 $U = \mathcal{P}(V), \mathcal{I} = \{S \in U : S \text{ forms } j\text{-forest}, j \geq k\}$, $\mathcal{I}$ 的遗传性显然, 交换性已由 (1) 给出. 所以 $\mathcal{M} = (U, \mathcal{I})$ 是拟阵. 上述算法是标准的拟阵贪心.

**复杂度分析**: 若以并查集维护连通性, 设 $|V| = n$, $|E| = m$, 则复杂度明显是 $\mathcal{O}(m \log m)$, 瓶颈在于排序.

**3.** (Clock Tree Synthesis)

**算法设计** (只计算最小代价值, $w_T(x)$ 表示 $x$ 的父边权值):

1 **function** $\text{solve}(u, T)$ :
2     **if** $u \in \text{leaf}_T$ **then**
3         **return** $(0, 0)$
4     $\{x, y\} \leftarrow \text{son}_T(u)$
5     $(\ell_x, c_x) \leftarrow \text{solve}(x, T)$
6     $(\ell_y, c_y) \leftarrow \text{solve}(y, T)$
7     $(\ell_x, \ell_y) \leftarrow (\ell_x + w_T(x), \ell_y + w_T(y))$
8     $\ell \leftarrow \max\{\ell_x, \ell_y\}$
9     **return** $(\ell, c_x + c_y + 2\ell - \ell_x - \ell_y)$

**正确性分析**: 显然 zero skew 树的任意子树都 zero skew. 设 $(\ell, c) = \text{solve}(u, T)$, 则 $\ell$ 是将 $u$ 子树调整为 zero skew 时可能的最小高度, $c$ 是最小代价. 在二叉树上结构归纳:

- 对叶子 $u$, 显然 $\ell = c = 0$.
- 对非叶结点 $u$ 和它的两个孩子 $x, y$, 在执行代码第 7 行后, 不妨设 $\ell_x \geq \ell_y$. 由 $\ell_x$ 的最小性, 我们至少需要将 $y$ 子树的树高增加 $\Delta := \ell_x - \ell_y$. 而代价最小的增加方式就是令 $w_T(y) \leftarrow w_T(y) + \Delta$, 于是 $c = c_x + c_y + (\ell_x - \ell_y)$, 也即是算法中的返回值. 这样, $(\ell, c)$ 归纳地满足上述性质.

**复杂度分析**: 明显是 $\mathcal{O}(|T|)$.

**4.** (Set Partition)

**解的表示方式**: 用一个长度为 $n$ 的二进制向量 $x$ 表示划分, $x_i = 1$ 表示第 $i$ 个元素属于 $V_1$, $x_i = 0$ 表示属于 $V_2$. 目标是最小化 $|\sum_{i:x_i=1} c_i - \sum_{i:x_i=0} c_i|$.

**解空间**: 所有 $2^n$ 种可能的划分.

**搜索策略**: 深度优先搜索 (DFS). 将元素按成本降序排列, 依次决定每个元素的归属.

**分支条件**: 对于当前元素 $i$, 有两个分支: 放入 $V_1$ 或放入 $V_2$.

**代价函数**: $f(S_1) = |\sum_{i \in S_1} c_i - \sum_{i \notin S_1} c_i|$.

**界的维护**: 维护当前最优值 $best$. 对于部分解 $S_1$ (已决定前 $k$ 个元素), 计算下界 $\ell(S_1, k)$ 如公式所示. 若 $\ell(S_1, k) \geq best$, 则剪枝; 否则继续分支. 当得到一个完整解时, 更新 $best = \min(best, f(S_1))$.

**分支策略**: 按 $c$ 降序排列元素, 每次决策当前元素是否 $S_1$.

**定界方法**: 设处理完了第 $k$ 个元素, $s := \sum_{i \in S_1} c(i)$, 预处理 $t_k := \sum_{i=k+1}^{n} c(i)$, 元素总和 $T = t_0$, 则有答案下界

$$\ell(S_1, k) = \max\{0, T - 2(s + t_k), 2s - T\}.$$

即假设剩下的 $n - k$ 个元素能够让 $S_1$ 增加 $[0, s_k]$ 中的任意一个整数.

**最坏复杂度**: 当 $c(i) = 2^{n-i}$, 在不加任何启发式分支策略的情况下, 搜索可能遍历所有取法, 复杂度 $\mathcal{O}(2^n)$.

**5.** (Minimizing Kendall tau distance)

**解的表示方式**: 用排列 $S = (s_1, s_2, \ldots, s_n)$ 表示聚合排名, 其中 $s_i$ 是排名第 $i$ 的元素.

**解空间**: 所有 $n!$ 种排列.

**搜索策略**: 深度优先搜索, 按排名位置 $1$ 到 $n$ 依次决定每个位置上放置哪个元素.

**分支条件**: 对于当前排名位置 $i$, 从尚未使用的元素中选择一个放置.

**代价函数**: $f(S) = K(S; P_1, \ldots, P_m) = \sum_{k=1}^{m} |\{(i,j) : \mathrm{rank}(i, P_k) < \mathrm{rank}(j, P_k) \wedge \mathrm{rank}(i, S) > \mathrm{rank}(j, S)\}|$.

**界的维护**: 维护当前最优值 $best$. 对于部分解 $S[1:k]$ (即前 $k$ 个位置已确定), 计算下界 $\ell(S[1:k])$ 如公式所示. 若 $\ell(S[1:k]) \geq best$, 则剪枝; 否则继续分支. 当得到一个完整排列时, 更新 $best = \min(best, f(S))$.

**分支策略**: 依次决定 $S_i$.

**定界方法**: 决定 $S[1:k]$ 后, 所有 $\{i,j\} \cap S[1:k] \neq \varnothing$ 的贡献对 $(i,j)$ 都能直接计算, 设它们对距离的贡献为 $d_k$, 剩下元素恰好构成 $n-k$ 规模的子问题, 记为 $P'_1, \cdots, P'_m$, 不妨 $P'_{m+1} = P'_1$, 则有答案下界

$$\ell(S[1:k]) = d_k + \frac{1}{2} \sum_{i=1}^{m} K(P'_i, P'_{i+1}).$$

**最坏复杂度**: 若所有 $P$ 相同, 在不加任何启发式分支策略的情况下, 搜索可能 (逆字典序地) 遍历所有排列, 复杂度 $\mathcal{O}(n! \cdot \mathrm{poly}(n,m))$, 通过合理的预处理可以做到 $\mathcal{O}(n! \cdot n)$, 添加 $S[1:k]$ 对应子问题的记忆化可以做到 $\mathcal{O}(2^n n)$.

**6.** (Circuit Routing)

**解的表示方式**: 从起点 $a$ 到终点 $b$ 的一条路径, 由一系列移动方向 (上, 下, 左, 右) 组成.

**解空间**: 所有从 $a$ 到 $b$ 且不经过阻塞格子的合法路径.

**搜索策略**: 使用优先队列 (A* 算法) 进行最佳优先搜索. 将起点加入优先队列, 每次取出估价函数最小的节点进行扩展.

**分支条件**: 从当前节点向四个方向移动, 如果相邻格子未被阻塞且未访问过 (或找到更短路径), 则生成新分支.

**代价函数**: 路径长度, 即从起点到当前节点经过的步数.

**界的维护**: 维护当前找到的最短路径长度 $best$ (初始化为无穷大). 对于每个节点 $(x,y)$, 使用启发函数 $h(x,y) = |x - x_b| + |y - y_b|$ 作为剩余路径长度的下界. 当前路径长度 $g(x,y)$ 加上 $h(x,y)$ 若大于等于 $best$, 则剪枝 (即不扩展该节点). 当第一次到达终点 $b$ 时, 得到的就是最优解, 可以立即停止.

**分支策略**: 使用 BFS, 决策当前向上/右/下/左移动.

**定界方法**: 由于第一次到达 $b$ 时得到的一定是最优答案, 不必再设计定界; 或者令 $\ell(x,y) = |x - x_b| + |y - y_b|$ 并使用优先队列进行 A* 搜索, 但在无权图上意义不大; 再或者用 $n \times m$ 个队列代替优先队列进行 A* 搜索.

**最坏复杂度**: 遍历到所有格子, $\mathcal{O}(nm)$. 如果一定要 A* 且使用优先队列, 由于搜索轮廓大小是 $\mathcal{O}(n+m)$ 的, 复杂度 $\mathcal{O}(nm\log(n+m))$.